Plug and Play

Modular Semantics for Quantum Programming Languages

Jacob Trevor
j.trevor.1@research.gla.ac.uk

School of Computer Science, University of Glasgow, Glasgow

https://github.com/jaketrevor/qpl-semantics

A Tale of Two Semantics

Gate Matrix (GM) Semantics

Consider the following program snippet:

```
X 0; X 0
```

What does this program *mean?*

Gate Matrix (GM) Semantics

Consider the following program snippet:

```
X 0; X 0
```

What does this program *mean?*

$$\mathcal{G}_N[\![-]\!] : \ ? \to \mathbb{C}^{2^N \times 2^N}$$

$$\mathcal{G}_N[\![G\,\vec{q}]\!] = \pi_N^{-1}\vec{q} \cdot \overset{N}{\uparrow} G \cdot \pi_N \vec{q}$$

$$\mathcal{G}_N[\![a; b]\!] = \mathcal{G}_N[\![b]\!] \cdot \mathcal{G}_N[\![a]\!]$$

Gate Matrix (GM) Semantics

Consider the following program snippet:

```
X 0; X 0
```

What does this program *mean?*

$$\mathcal{G}_N[\![-]\!] : \ ? \to \mathbb{C}^{2^N \times 2^N}$$

$$\mathcal{G}_1[\![\texttt{X 0; X 0}]\!] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Advantages of GM Semantics:

- Relatively simple to define (See above)
- Easy(ish) to explain
- Analytically, very nice
  - Full abstraction
  - Constructive, decidable equality
  - Directly compare matrices; 2^{2n} comparisons

In fact, lean can just do the check for you, if you set it up correctly:

```
53  example : [Gate 1 ℤ]Sem⟦ X0 ;;; X0 ⟧ = Sem⟦emptyProg⟧ := by
54    native_decide
```

A (slightly) more complicated program

```
X 0;
X 0;
Meas 0
```

We can't give this a GM semantics - we need something *stronger*

Density Operator Transformer (DOT) Semantics

$$\mathcal{D}_N[\![-]\!] : \ ? \to \rho_N \to \rho_N$$

$$\mathcal{D}_N[\![a; b]\!] = \mathcal{D}_N[\![b]\!] \circ \mathcal{D}_N[\![a]\!]$$

$$\mathcal{D}_N[\![G\,\vec{q}]\!] = \text{let } G' := \pi_N^{-1}\vec{q} \cdot \overset{N}{\uparrow} G \cdot \pi_N\vec{q}' \text{ in } \lambda\rho.G'\,\rho G'^{\dagger}$$

$$\mathcal{D}_N[\![\texttt{Meas q}]\!] = \lambda\rho. \sum_{i \in \text{ t, f}} P_i \rho P_i^{\dagger}$$

Comparison

DOT semantics is a little less nice

- Direct equality check no longer works
  - Must instead check a *basis*
  - $\sim 2^{2n}$ vs $\left(2^{2n}\right)^2$ comparisons i.e. quadratic slowdown
  - hard to mechanise

But!

- We can handle programs with measurement

Comparison

DOT semantics is a little less nice

- Direct equality check no longer works
  - Must instead check a *basis*
  - $\sim 2^{2n}$ vs $\left(2^{2n}\right)^2$ comparisons i.e. quadratic slowdown
  - hard to mechanise

But!

- We can handle programs with measurement

A tale as old as time: to handle more complicated languages, we need more complicated semantics

The baby and the bathwater

- Do we have to give up the nice analytic properties of our simple semantics to be able to work in more expressive languages?

The baby and the bathwater

- Do we have to give up the nice analytic properties of our simple semantics to be able to work in more expressive languages?

I say: No!

- Cut the language up into *features**
- Give each a *weakest semantics**
- Define a way to *glue* features together
- In a way that lets us derive the semantics of the result

* we'll define this in a moment

Fragments and Feature sets

Language Features

Take a look at this program again:

```
X 0;
X 0;
Meas 0
```

Language Features

Take a look at this program again:

```
X 0;
X 0;
Meas 0
```

Two distinct parts:

- `Meas 0` requires DOT ($\mathcal{D}$) semantics…

Language Features

Take a look at this program again:

```
X 0;
X 0;
Meas 0
```

Two distinct parts:

- `Meas 0` requires DOT ($\mathcal{D}$) semantics…
- but `X 0` can be given GM ($\mathcal{G}$) semantics

Language Features

Take a look at this program again:

```
X 0;
X 0;
Meas 0
```

Two distinct parts:

- `Meas 0` requires DOT ($\mathcal{D}$) semantics…
- but `X 0` can be given GM ($\mathcal{G}$) semantics

Idea: Cut this language up into “little languages”

Prior work

- What follows is essentially a specialisation of Wouter Swierstra's DTalC [1]
- Since I am mechanising it, a bit of extra trickery required to make the typechecker happy
  - i.e. containers
- This is a technique adapted from Jonas Maier's 2025 Masters Thesis (LRalC) [2]

Features

- A language *feature* is meant to capture “doing one kind of thing”
  - À la the little languages of Jon Bentley [3]

Features

- A language *feature* is meant to capture "doing one kind of thing"
  - À la the little languages of Jon Bentley [3]

So:

Unitary App (U) = { $G\ \vec{q}$ }
Effectful Measurement (M) = { Meas q }

Features

- A language *feature* is meant to capture "doing one kind of thing"
  - À la the little languages of Jon Bentley [3]

So:

Unitary App (U) = { $G\ \vec{q}$ }
Effectful Measurement (M) = { Meas q }

- Each feature becomes a *container*

Containers in 30 seconds

- Think of a container as consisting of:

1. A constant part
2. A number of holes

- The extension of a container into Y tells you how to fill the holes with Ys
- We are interested in extending the container into more copies of itself…
- Called the fixed point of a container

Constant Data

```
-- α the type of matrix entries
-- i the bitdwidth of the circuit

inductive uData i α  where
  | app : (h_size : n ≤ i) -- proof the gate fits
        → Gate n α         -- the gate to apply
        → PartialPerm n i  -- the list of targets
        → uData i α

inductive mData i (α : Type) where
  | meas : Fin i → mData i α
```

Containers

```
-- Command D n = "A command with constant data D and n holes"
abbrev Command (data : Type) (arity : Nat) : Container
  := {shape := data, pos := fun _ ⇒ Fin arity}


def U := Command uData 0
def M := Command mData 0

def Seq := Command Unit 2 -- sequence two commands
```

Key idea

- We can add feature containers together by taking their *coproduct* ($\oplus$)
  - The result is another container
- Our AST is now the *fixed point* of some container

Key idea

- We can add feature containers together by taking their *coproduct* ($\oplus$)
  - The result is another container
- Our AST is now the *fixed point* of some container

e.g. the AST of circuit, the language of our second example, can be defined as:

$$\text{Circuit} := (\text{Seq} \oplus U \oplus M).\text{fix}$$

Semantics of Containerised ASTs

- Define semantics via the container recursor

$$\frac{\mathcal{X}[\![A.\text{ext}\ \mathcal{X}]\!]}{\mathcal{X}[\![A.\text{fix}]\!]}$$

- Sort-of like a post-order tree traversal
  - Given that we evaluate the children of this node…
  - i.e for each hole, we have an $\mathcal{X}$
- So just define the semantics of $A.\text{ext}\ \mathcal{X}$

Deriving Semantics

- If A and B have semantics in the same terms, then their coproduct does too

$$\frac{\mathcal{X}[\![A.\text{ext } \mathcal{X}]\!] \qquad \mathcal{X}[\![B.\text{ext } \mathcal{X}]\!]}{\mathcal{X}[\![(A \oplus B).\text{ext } \mathcal{X}]\!]}$$

- Via pattern-matching on the shape, and applying the corresponding instance
- See Lean for details

Subsumption

Subsumption

- Even with a finite collection of features, there's an infinite variety of feature sets
  - A, $A \oplus A$, $A \oplus A \oplus A$, ...
- It's also clear that A is somehow inside $A \oplus B$
  - e.g. {`X 0; X 0`} is inside `X 0; X 0; M 0`
- How do we formalise this notion?

Subsumption

A language *subsumes* another if there's a semantics preserving transformation from one to the other

$$A \preceq [\mathcal{X}]B \Leftrightarrow \exists(f : A \to B).\mathcal{X}[\![x]\!] = \mathcal{X}[\![f\,x]\!]$$

Subsumption

A language *subsumes* another if there's a semantics preserving transformation from one to the other

$$A \preceq [\mathcal{X}]B \Leftrightarrow \exists(f : A \to B).\mathcal{X}[\![x]\!] = \mathcal{X}[\![fx]\!]$$

When the semantics in question is obvious, I'll omit the $[\mathcal{X}]$ e.g.

$$A \preceq B$$

Isomorphisms

- If there's a semantics preserving function in both directions, then the languages can express *exactly the same things*
- i.e. they are isomorphic.
- We write

$$A \cong B \Leftrightarrow A \preceq B \wedge B \preceq A$$

Isomorphisms

- If there's a semantics preserving function in both directions, then the languages can express *exactly the same things*
- i.e. they are isomorphic.
- We write

$$A \cong B \Leftrightarrow A \preceq B \wedge B \preceq A$$

- If $A \preceq B$ but not $B \preceq A$ then we write $A \prec B$
- intuitively, B can express things A cannot

Normal form lemmas

$(1) A.\mathrm{fix} \preceq (A \oplus A).\mathrm{fix}$

$(2) (A \oplus A).\mathrm{fix} \preceq A.\mathrm{fix}$

$(3) (A \oplus B).\mathrm{fix} \preceq (B \oplus A).\mathrm{fix}$

Normal form lemmas

$$(1) A \preceq A \oplus A \qquad (2) A \oplus A \preceq A$$

$$(3) A \oplus B \preceq B \oplus A$$

Normal form lemmas

$$(1) A \preceq A \oplus A \qquad (2) A \oplus A \preceq A$$

$$(3) A \oplus B \preceq B \oplus A$$

- (1) and (2) say $A \cong A \oplus A$
  - Need only consider A; $A \oplus A$ &c. are equivalent
- (3) gives us $A \oplus B \cong B \oplus A$

Normal form lemmas

$$(1) A \preceq A \oplus A \qquad (2) A \oplus A \preceq A$$

$$(3) A \oplus B \preceq B \oplus A$$

- (1) and (2) say $A \cong A \oplus A$
  - Need only consider A; $A \oplus A$ &c. are equivalent
- (3) gives us $A \oplus B \cong B \oplus A$
- Together, these give us a normal form for feature sets
  - Distinct features only
  - “Lexicographic” ordering

Embedding lemma

$$A \preceq A \oplus B$$

- **Intuition:** There's an obvious way to embed A into a larger feature set

Embedding lemma

$$A \preceq A \oplus B$$

- **Intuition:** There's an obvious way to embed A into a larger feature set
- **Implication:** Programs in smaller feature sets are valid in larger ones
  - i.e. *syntactic subsumption*

Embedding lemma

$$A \preceq A \oplus B$$

- **Intuition:** There's an obvious way to embed A into a larger feature set
- **Implication:** Programs in smaller feature sets are valid in larger ones
  - i.e. *syntactic subsumption*
- **Reminder:** $\preceq$ means we have a SPF from A to $A \oplus B$
  - So for any program in A, we can generate the matching program in $A \oplus B$
  - e.g. $\text{emb}_{U \oplus M}\{-\}_U$ embed the program "$-$" in U into $U \oplus M$

Aside:

- I've omitted the proof for these, but there's a verified version in lean
- the proofs aren't particularly hard or interesting.

Summary so far:

- We can cut a language up into fragments
- We can add the fragments together as we wish
- So long as they all have semantics in the same terms, we can derive semantics the larger products

Summary so far:

- We can cut a language up into fragments
- We can add the fragments together as we wish
- So long as they all have semantics in the same terms, we can derive semantics the larger products

What if I don't want to give semantics in the same terms all the time?

Semantic strength and weakness

Semantic strength

Fact: We can derive a DOT ($\mathcal{D}_N[\![-]\!]$) semantics from the GM ($\mathcal{G}_N[\![-]\!]$) semantics

$$\mathcal{G}_N[\![-]\!] : \ ? \to \text{Gate } N$$
$$\mathcal{D}_N[\![-]\!] : \ ? \to \rho_N \to \rho_N$$

Semantic strength

Fact: We can derive a DOT ($\mathcal{D}_N[\![-]\!]$) semantics from the GM ($\mathcal{G}_N[\![-]\!]$) semantics

$$\mathcal{G}_N[\![-]\!] : \ ? \to \text{Gate } N$$
$$\mathcal{D}_N[\![-]\!] : \ ? \to \rho_N \to \rho_N$$
$$\mathcal{D}_N[\![x]\!] = \lambda\rho.\ (\mathcal{G}_N[\![x]\!])\rho\big(\mathcal{G}_N[\![x]\!]^\dagger\big)$$

Semantic strength

Put another way, we have a function:

$$\begin{aligned} \text{cast} &: \text{Gate } N \to \rho_N \to \rho_N \\ \text{cast} \quad G \quad &= \lambda\rho.\ (G)\rho\left(G^\dagger\right) \end{aligned}$$

$$\mathcal{D}_N[\![x]\!] = \quad \text{cast} \circ \mathcal{G}_N$$

Semantic strength

Put another way, we have a function:

$$\begin{aligned} \text{cast} &: \text{Gate } N \to \rho_N \to \rho_N \\ \text{cast} \quad G \quad &= \lambda\rho.\ (G)\rho\left(G^\dagger\right) \end{aligned}$$

$$\mathcal{D}_N[\![x]\!] = \quad \text{cast} \circ \mathcal{G}_N$$

- $\mathcal{D}_N$ is derivable from $\mathcal{G}_N$, so $\mathcal{G}_N$ is weaker than $\mathcal{D}_N$ ($\mathcal{G}_N \sqsubseteq \mathcal{D}_N$)
  - Note $\mathcal{G}_N \sqsubseteq \mathcal{D}_N$ is the same way around as the function $\text{Dom}(\mathcal{G}_N) \to \text{Dom}(\mathcal{D}_N)$

Aside: Sensible Functions

- cast needs to be a *sensible* choice of function

Aside: Sensible Functions

- cast needs to be a *sensible* choice of function
- We shouldn't lose information in the transformation

Aside: Sensible Functions

- cast needs to be a *sensible* choice of function
- We shouldn't lose information in the transformation
- Therefore, it should be an *embedding* i.e.

$$\exists \downarrow, \downarrow \circ \text{cast} = \text{id}$$

Aside: Sensible Functions

- cast needs to be a *sensible* choice of function
- We shouldn't lose information in the transformation
- Therefore, it should be an *embedding* i.e.

$$\exists \downarrow, \downarrow \circ \mathrm{cast} = \mathrm{id}$$

- Proving that the cast function above is a proper embedding has proved to be tricker than expected
- Only works up to a global phase
- Which is OK, but harder to mechanise… WIP

Note that the same is not true in reverse:

- DOT semantics ($\mathcal{D}_N$) can express more than GM semantics ($\mathcal{G}_N$)
- We can't embed $\mathcal{D}_N$ into $\mathcal{G}_N$
- Therefore $\mathcal{G}_N \sqsubset \mathcal{D}_N$

Weakest semantics

A language L has a weakest semantics in $\mathcal{X}$ iff for any other semantics of L $\mathcal{Y}$, $\mathcal{X} \sqsubseteq \mathcal{Y}$

Weakest semantics

A language L has a weakest semantics in $\mathcal{X}$ iff for any other semantics of L $\mathcal{Y}$, $\mathcal{X} \sqsubseteq \mathcal{Y}$

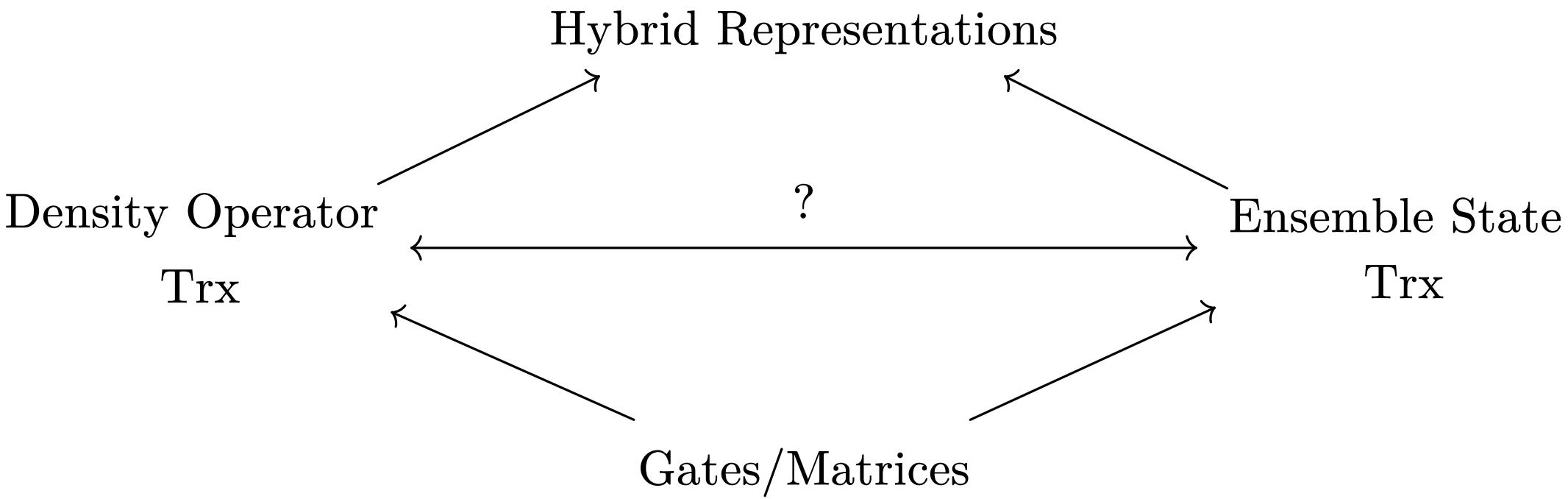
- Why care?

Weakest semantics

A language L has a weakest semantics in $\mathcal{X}$ iff for any other semantics of L $\mathcal{Y}$, $\mathcal{X} \sqsubseteq \mathcal{Y}$

- Why care?
- This makes $\mathcal{X}$ a *complete description* of L
- $\mathcal{X}$ is sufficient to derive any other semantics we might want
- Need only define $\mathcal{X}$, get everything else "for free"

Formalising some folklore



Caveat: The exact shape of this is still in question

Big Result:

- Give semantics to each feature in the weakest form
- Derive semantics of feature sets in their weakest form
  - i.e. the least upper bound of the features w.s.
  - Works so long as we are working in a CPO of semantic domains

Big Result:

- Give semantics to each feature in the weakest form
- Derive semantics of feature sets in their weakest form
  - i.e. the least upper bound of the features w.s.
  - Works so long as we are working in a CPO of semantic domains

But!

- This isn't just a neat trick
- By congruence, equality judgements in simpler semantics *must hold* in more complex ones

i.e. if $\mathcal{X}[\![a]\!] = \mathcal{X}[\![b]\!]$ and $X \sqsubseteq Y \Leftrightarrow \mathcal{Y} = \text{cast} \circ \mathcal{X}[\![-]\!]$ then $\mathcal{Y}[\![a]\!] = \mathcal{Y}[\![b]\!]$

Concretely…

- If I know that $\mathcal{G}_N[\![\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{G}_N[\![\{\}_U]\!]$
  - where $\{\}$ is the empty program
  - prove easily by calculating the matrix representation of each side

Concretely…

- If I know that $\mathcal{G}_N[\![\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{G}_N[\![\{\}_U]\!]$
  - where $\{\}$ is the empty program
  - prove easily by calculating the matrix representation of each side
- Then I know that $\mathcal{D}_N[\![\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{D}_N[\![\{\}_U]\!]$

Concretely…

- If I know that $\mathcal{G}_N[\![\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{G}_N[\![\{\}_U]\!]$
  - where $\{\}$ is the empty program
  - prove easily by calculating the matrix representation of each side
- Then I know that $\mathcal{D}_N[\![\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{D}_N[\![\{\}_U]\!]$
- And I can then conclude that

$$\mathcal{D}_N[\![\mathrm{emb}_{U\oplus M}\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{D}_N[\![\mathrm{emb}_{U\oplus M}\{\}_U]\!]$$

Concretely…

- If I know that $\mathcal{G}_N[\![\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{G}_N[\![\{\}_U]\!]$
  - where $\{\}$ is the empty program
  - prove easily by calculating the matrix representation of each side
- Then I know that $\mathcal{D}_N[\![\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{D}_N[\![\{\}_U]\!]$
- And I can then conclude that

$\mathcal{D}_N[\![\text{emb}_{U\oplus M}\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{D}_N[\![\text{emb}_{U\oplus M}\{\}_U]\!]$

- And thus show that $\mathcal{D}_N[\![\{\texttt{X 0; X 0; M 0}\}_{U\oplus M}]\!] = \mathcal{D}_N[\![\{\texttt{M 0}\}_{U\oplus M}]\!]$
- Without ever having to look at a DOT!

Verifying certain kinds of program rewriting is now trivial!

Future work & Conclusions

Strengthening: problem

Naïvely strengthening along $X \sqsubseteq Y$ yields:

$$\frac{X \sqsubseteq Y \qquad \mathcal{X}[\![A.\text{ext } \mathcal{X}]\!]}{\mathcal{Y}[\![A.\text{ext } \mathcal{X}]\!]}$$

- Which isn't quite right
  - remember we need $\mathcal{Y}[\![A.\text{ext } \mathcal{Y}]\!]$ to define $\mathcal{Y}[\![A.\text{fix}]\!]$
- There is a fix…

Strengthening: fix

Split your features into:

- "terminal features" — with $n = 0$ holes, and set semantics in $\mathcal{X}$
  - Yields $\forall \beta, \mathcal{X}[\![A.\text{ext } \beta]\!]$
  - Ensure $\mathcal{X}$ is the weakest semantics, and we're done
- "structural features" — with $n > 0$ holes, and *parametric* semantics
  - i.e. semantics is defined $\forall \alpha, \alpha[\![A.\text{ext } \alpha]\!]$
  - No strengthening needed
  - You're allowed to put restrictions (e.g. typeclass constraints) on α

Onwards

- A few definitions still use `sorry`

Onwards

- A few definitions still use `sorry`
- Next Stop: Hybrid languages
  - Semantics becomes much more interesting when classical co-processing is involved
  - Should be some nice strength & subsumption results to come
  - Dynamic qubit allocation is of particular interest

Onwards

- If you're thinking 'Weakest Semantics' sounds an awful lot like 'Initial Model'

Onwards

- If you're thinking 'Weakest Semantics' sounds an awful lot like 'Initial Model'
  - I agree!
  - Someone should think about it
  - Not me (for now)

Applications

- Clear applications to *program optimisation*
  - Every equivalence between two programs induces a SPF
  - If this SPF is an “improvement”
  - It’s a verified optimisation
  - Valid even in more complex languages
  - See last slide of previous section for an example
  - Mechanise/automate the generation of such functions

Bibliography

[1] W. Swierstra, “Data types à la carte,” *J. Funct. Program.*, vol. 18, no. 4, pp. 423–436, 2008, doi: 10.1017/S0956796808006758.

[2] J. Maier, “Logical relations à la carte,” Master's thesis, 2026. [Online]. Available: https://ethz.ch/content/dam/ethz/special-interest/infk/inst-pls/plf-dam/documents/StudentProjects/MasterTheses/2025-Jonas-Thesis.pdf

[3] J. Bentley, “Programming pearls: little languages,” *Commun. ACM*, vol. 29, no. 8, pp. 711–721, Aug. 1986, doi: 10.1145/6424.315691.

That's all folks

Lean code repository:

https://github.com/jaketrevor/qpl-semantics

Contact me:

J.Trevor.1@research.gla.ac.uk

Any Questions?